

The Sorter

Triage Threads by Priority

Gemini reads your inbox and decides what actually needs your attention.

URGENT

NEEDS REPLY

FYI

IGNORE

MS 1.3

Session

Thursday

Day

Prompt Engineering

Skill

Triage Function

Deliverable

Windows

OS

Sprint 1 — Where We Are

Three micro-skills. One pipeline.

Tuesday
MS 1.1

✓ The Loop

MCP architecture. Cline + Gemini running. Filesystem server connected. Agent read a real file.

Wednesday
MS 1.2

✓ The Inbox

Gmail MCP server. OAuth credentials. Agent listed, searched and fetched real email threads.

TODAY
MS 1.3

The Brain

Feed thread metadata to Gemini. Get back URGENT / NEEDS-REPLY / FYI / IGNORE labels. Build the reusable triage function.

Saturday
Sprint Build

The Engine

Wire all three together: pull 20 real threads → classify each → output a ranked digest. Public ship.

After today: you have all three pieces. Saturday you assemble them

What Is Triage?

The decision your brain makes 50 times a day — automatically.



What YOU do now

- Open email
- Read subject + sender
- Scan first line
- Decide: act / reply / ignore
- Repeat × 50 threads
- Lose 45 minutes



~45 min/day of your best thinking



What your agent does

- ✓ **Pull thread metadata**
from Gmail MCP server
- ✓ **Format it as a prompt**
structured context for Gemini
- ✓ **Ask Gemini to classify**
URGENT / NEEDS-REPLY / FYI / IGNORE
- ✓ **Parse the labels**
extract structured output
- ✓ **Return ranked digest**
sorted by priority
- ⚡ **~3 seconds**

The 4 Priority Labels

Design these carefully — they shape every downstream decision.

URGENT

When: Needs action TODAY. Time-sensitive. From someone important.

e.g. Boss: client escalation needs response today

e.g. Calendar invite: meeting in 1 hour

NOT: Something marked urgent by the sender but is actually a newsletter

NEEDS REPLY

When: Requires a response, but not today. Within 24–48 hours.

e.g. Colleague: quick question about a project

e.g. Partner: scheduling a meeting next week

NOT: A chain where someone else is already handling it

FYI

When: Good to know. No action needed. File for reference.

e.g. Invoice: payment confirmation

e.g. Automated: your weekly report is ready

NOT: Newsletters or promotional content you never asked for

IGNORE

When: No value. Delete or archive without reading.

e.g. Newsletter from a site you barely remember subscribing to

e.g. Recruiter cold outreach

NOT: Automated alerts you actually need — check sender carefully

Giving Gemini the Right Context

What you feed the model shapes what it returns.

For each thread, we pass this metadata to Gemini:

Thread Input

```
{
  "from": "boss@company.com",
  "subject": "URGENT: Client escalation
             needs response today",
  "snippet": "Hi, the client called
             again this morning...",
  "date": "2024-06-11T09:14:00Z",
  "thread_id": "18f3a2b..."
}
```

What we don't pass (yet)

- ✗ Full email body — too many tokens
- ✗ Attachments — not needed for triage
- ✗ HTML/quoted text — noise

Why this is enough

- ✓ Sender + subject = 80% of the signal
- ✓ Snippet catches urgency cues
- ✓ Date flags time-sensitive threads
- ✓ Keeps token cost near zero

The Triage Prompt

The instruction that turns raw threads into decisions.

System prompt — paste this into your triage function:

```
You are an expert email triage assistant.
You will receive a list of email threads as JSON.
For each thread, classify it into exactly one priority:

URGENT      — needs action today, time-sensitive
NEEDS-REPLY — requires a response within 24-48 hours
FYI         — informational, no action needed
IGNORE      — no value, can be archived

Rules:
- Base your decision on sender, subject, and snippet only
- Do not assume urgency just because the sender says "urgent"
- Return ONLY a JSON array — no explanation, no preamble
```

Output format:

```
[
  { "thread_id": "...", "priority": "URGENT", "reason": "one sentence" },
  ...
]
```

Role framing

Telling Gemini it's an expert sets the tone of judgement

Exact labels

Hard-coding the four labels prevents Gemini inventing new ones

JSON output only

No prose — makes parsing reliable and fast

Reason field

One-sentence reason makes the output auditable and trustworthy

Building the Triage Function

The reusable function you'll call all month.

Create `triage.py` in your project folder:

```
import google.generativeai as genai
import json

genai.configure(api_key="YOUR_GEMINI_API_KEY")
model = genai.GenerativeModel("gemini-2.0-flash")

SYSTEM_PROMPT = """...(paste the triage prompt here)"""

def triage_threads(threads: list[dict]) -> list[dict]:
    """
    Takes a list of thread dicts (from Gmail MCP).
    Returns list of {thread_id, priority, reason}.
    """
    user_message = json.dumps(threads, indent=2)

    response = model.generate_content(
        SYSTEM_PROMPT + "\n\nThreads to classify:\n" + user_message
    )

    raw = response.text.strip()
    # Strip markdown code fences if Gemini adds them
    if raw.startswith("```"):
        raw = raw[3:-3]
```

Install & Test It

Get it working before wiring it to Gmail.

1. Install the Gemini SDK

In your VS Code terminal:

```
pip install google-generativeai
```

2. Get your Gemini API key

aistudio.google.com → Get API Key → copy it into triage.py

(Same key as Cline — reuse it)

3. Test with sample data

```
# test_triage.py
from triage import triage_threads

sample = [
    {"thread_id": "1", "from": "boss@co.com",
     "subject": "URGENT: client call today",
     "snippet": "They called again..."},
    {"thread_id": "2", "from": "news@medium.com",
     "subject": "10 articles this week",
     "snippet": "Top stories for you..."}
]

result = triage_threads(sample)
print(result)
```

Expected output →

```
[ { "thread_id": "1", "priority": "URGENT", "reason": "Boss requesting urgent
client action today." },
  { "thread_id": "2", "priority": "IGNORE", "reason": "Promotional newsletter, no
action required." } ]
```


Wire It to Your Real Inbox

Gmail MCP → triage function → ranked digest.

Now replace the sample data with live threads from Cline:

```
# Ask Cline to do this in one go:

1. Use the Gmail MCP server to fetch the last 20 threads
   (return as JSON: thread_id, from, subject, snippet, date)

2. Pass that list to the triage_threads() function in triage.py

3. Sort the results by priority order:
   URGENT first → NEEDS-REPLY → FYI → IGNORE

4. Print a clean digest showing:
   [PRIORITY] From: ... Subject: ... Reason: ...
```

Output looks like this →

URGENT	boss@company.com	Client escalation today	Time-sensitive action from manager
NEEDS-REPLY	teammate@company.com	Quick question about Q3 deck	Colleague waiting on your input
FYI	billing@software.com	Your invoice is ready	Payment confirmation, no action

Prompt Engineering for Triage

Small changes to the prompt = big changes in accuracy.

Be explicit about boundaries

✗ Classify this email as urgent or not urgent.

✓ Classify as URGENT only if action is needed TODAY. "Urgent" in the subject line alone is not enough.

Why: Vague instructions lead to over-classification. Senders cry wolf — Gemini needs the rule.

Force JSON output — always

✗ Tell me the priority of each email.

✓ Return ONLY a JSON array. No explanation. No preamble. No markdown.

Why: Prose responses break your parser. The word ONLY is load-bearing.

Include a reason field

✗ `{ "thread_id": "1", "priority": "URGENT" }`

✓ `{ "thread_id": "1", "priority": "URGENT", "reason": "Boss requires client response today." }`

Why: A reason makes errors visible. When the model is wrong, the reason tells you why — so you can fix the prompt.

Test edge cases before Saturday

✗ Only test with obvious emails (clear spam, clear urgent).

✓ Test with ambiguous cases: newsletters you might actually read, cold emails from important people.

Why: Edge cases expose prompt weaknesses. Better to find them in testing than in your live digest.

Saturday — Sprint Build 1

You now have everything you need.

The Triage Engine — what you're assembling on Saturday:



✓ Already Built

- Gmail MCP server connected (MS 1.2)
- triage_threads() function (today)
- Gemini API working
- OAuth token in place

🔨 To Build Saturday

- fetch_threads() — call Gmail MCP for 20 threads
- sort_by_priority() — order the classified output
- format_digest() — clean readable output
- Main runner — wire it all together

🏆 The Proof

- 20 real threads classified
- Screenshot of ranked digest
- LinkedIn post with #MyAIChiefOfStaff
- Micro-badge: The Switchboard Operator

What You Did Today

Sprint 1 micro-skills: complete.



01 Understand email triage — and why it's a solvable AI problem



02 Defined 4 priority labels: URGENT / NEEDS-REPLY / FYI / IGNORE



03 Wrote a triage system prompt with exact output format



04 Built triage_threads() — a reusable Python function



05 Wired it to your real inbox — 20 live threads classified by Gemini

Saturday — Sprint Build 1

The Triage Engine

Wire everything together. Pull 20 real threads. Classify all of them. Output a clean digest. Ship it publicly.

Tonight: tweak your triage prompt on edge cases. The better your prompt, the better your Saturday output.



Post your triage digest →
#MyAIChiefOfStaff

Sprint 1 progress:

MS 1.1 ✓

MS 1.2 ✓

MS 1.3 ✓

Sprint Build